Inception-of-Things (IoT)

A System Administration project (42/1337 school, subject v4.0). The goal is to get hands-on with **Kubernetes** through two lightweight distributions — **K3s** and **K3d** — provisioned with **Vagrant**, and to build a small **GitOps** pipeline with **Argo CD**.

This README only covers the **mandatory part** (Part 1, Part 2, Part 3). The bonus (Gitlab) is intentionally left out, as requested.

> Rule from the subject: everything must run inside a **virtual machine**, and each part's config files live in its own folder at the repo root: `p1/`, `p2/`, `p3/` (each containing a `scripts/` and a `confs/` subfolder).

---

0. What you need to learn *before* starting

You don't need to master all of this up front — learn each topic right before the task that needs it — but knowing the map helps a lot.

Core concepts (needed for every part)

- **Linux basics & shell scripting** — you'll write provisioning/install `.sh` scripts.
- **Networking basics** — static IP addressing, host-only networks, what an interface (`eth0`, `enp0s8`…) is, `ip a` / `ifconfig`.
- **SSH** — key-based (passwordless) authentication.
- **YAML syntax** — every Kubernetes manifest is YAML.

Kubernetes concepts (needed for Part 1 & 2)

- What a **cluster**, **node**, **pod** are.
- **Deployment** (manages ReplicaSets/Pods) vs **ReplicaSet** vs **Pod**.
- **Service** (ClusterIP) — stable internal networking for pods.
- **Ingress** + **Ingress Controller** (K3s ships **Traefik** by default) — routes external HTTP traffic to services based on **Host header** or path.
- `kubectl` basics: `get`, `describe`, `apply -f`, `logs`, namespaces (`-n`).
- **K3s** itself: what makes it "lightweight" (single binary, embedded containerd, SQLite by default), server mode vs agent mode, the node token used to join agents.

Vagrant concepts (needed for Part 1)

- `Vagrantfile` syntax: `config.vm.define`, `config.vm.network "private_network", ip: ...`, `config.vm.provider`, `config.vm.provision "shell"`.
- Difference between **inline** provisioning and a **provisioning script file**.

Docker concepts (needed for Part 3)

- Images, containers, tags, and Docker Hub as a registry.
- Why K3d needs Docker (K3d runs a full K8s cluster *as Docker containers*, unlike K3s which installs directly on a VM/host as a systemd service).

GitOps / Argo CD concepts (needed for Part 3)

- **GitOps**: your Git repo is the single source of truth for what should be running in the cluster; a controller (Argo CD) continuously reconciles the live cluster to match Git.
- Argo CD **Application** custom resource: points to a Git repo + path + target namespace, and can sync manually or automatically.
- Image **tagging** (`v1`, `v2`) to represent two versions of the same app.

Suggested reading order: Vagrant Quick Start → Kubernetes "Concepts" docs → K3s docs (Rancher/SUSE) → Docker basics (if needed) → K3d docs → Argo CD "Getting Started".

Part 1 — K3s and Vagrant (`p1/`)

Goal: two VMs created by Vagrant; K3s installed as **server** on one and **agent** on the other; reachable over static IPs.

Requirements recap

- 2 VMs, minimal resources (1 CPU / 512–1024 MB RAM), **latest stable** distro of your choice.
- Hostnames: `<login>S` (server) and `<login>SW` (server-worker).
- Static IPs on the private network: `192.168.56.110` (S) and `192.168.56.111` (SW).
- Passwordless SSH into both machines.
- K3s installed in **controller (server)** mode on S, **agent** mode on SW.
- `kubectl` usable to verify (`k get nodes -o wide` should show both nodes `Ready`).

Step-by-step tasks

1. **Pick a distro** (Debian, Rocky/AlmaLinux, Ubuntu server, etc.) and find its latest Vagrant box on Vagrant Cloud.
2. **Write the `Vagrantfile`** with two `config.vm.define` blocks (`wilS`-style names using your own login), each with: - `hostname` - `private_network` with the fixed IP - provider tuning (memory/cpus) - a `shell` provisioner pointing at a script in `p1/scripts/`
3. **Write `p1/scripts/server.sh`** (runs on S): install K3s in server mode (`curl -sfL https://get.k3s.io | sh -`), then copy/print the node token from `/var/lib/rancher/k3s/server/node-token` (you already save it to `p1/k3s-token` in your current setup — good).
4. **Write `p1/scripts/worker.sh`** (runs on SW): install K3s in agent mode, pointing at the server via `K3S_URL=https://192.168.56.110:6443` and `K3S_TOKEN=<token from step 3>`.

5. **Set up passwordless SSH** (key-based, e.g. via `install_ssh_keys.sh` as you already have).
6. **Test**: `vagrant up`, then from S run `k get nodes -o wide` — S must show role `control-plane,master`, SW must show as an agent node, both `Ready`, with the correct internal IPs.

(From your repo status, `p1/` already has a `Vagrantfile` and these scripts — worth re-verifying against this checklist rather than starting from scratch.)

Part 2 — K3s and three simple applications (`p2/`)

Goal: a **single** VM (`<login>S`, K3s server mode), running **3 web apps**, routed by **Ingress** based on the `Host` header hitting `192.168.56.110`.

Requirements recap

- `app1.com` → app1 (1 replica)
- `app2.com` → app2 (**3 replicas**)
- anything else (no matching host) → app3 (default backend)
- Ingress must be demonstrated live during defense (it's deliberately not screenshotted in the subject).

Step-by-step tasks

1. For each app, create in `p2/confs/appX/`: - `deployment.yml` — a simple web server image (nginx, httpd, or a tiny custom image); set `replicas: 3` only for app2. - `service.yml` — `ClusterIP` service exposing the deployment. - `ingress.yml` — an Ingress rule matching `Host: appX.com`, routed to the app's service.
2. For app3, make its Ingress rule the **default/catch-all** (no `host` restriction, or a default backend) so any other Host value lands there.
3. `kubectl apply -f` everything (or wire it into your provisioning script, as you appear to be doing with `p2/scripts/setup.sh`).
4. **Test** with curl using explicit Host headers, since you won't have real DNS: `curl -H "Host: app1.com" http://192.168.56.110 curl -H "Host: app2.com" http://192.168.56.110 curl http://192.168.56.110 # should fall through to app3`
5. Confirm app2 actually load-balances across its 3 pods (`k get pods` shows 3 replicas, and repeated curls may hit different pod names if your app reports its pod name — useful for the defense).

(Your repo already shows `p2/confs/app1`, `app2`, `app3` with deployment/service/ingress files and a working test per your commit messages — this section is mainly a checklist to confirm nothing's missing, e.g. the "default backend" behavior for app3.)

Part 3 — K3d and Argo CD (`p3/`)

Goal: no more Vagrant for this part — same VM, but now you install **Docker + K3d**, spin up a cluster with K3d, and build a **GitOps pipeline** with **Argo CD** that deploys an app from your public GitHub repo and can switch between two image versions (`v1` / `v2`).

Requirements recap

- Write a **script** that installs everything needed (Docker, K3d, kubectl, etc.) — you must be able to run/show it live during the defense.
- Create a K3d cluster.
- Two namespaces: `argocd` (for Argo CD itself) and `dev` (for the deployed app).
- A **public GitHub repo** containing your config files, with the group member's login in its name.
- The deployed app must exist in **two tagged versions** (`v1` and `v2`) — either:
- use Wil's pre-made app `wil42/playground` (Docker Hub, port 8888), **or**
- build/push your own app to a public Docker Hub repo, tagged `v1` and `v2` with a visible difference between them.
- You must be able to edit the GitHub repo (change the image tag), push, and have Argo CD sync the change into the cluster — then prove it via `curl` against the app.

Step-by-step tasks

1. **Understand K3s vs K3d first**: K3d = "K3s in Docker" — it runs cluster nodes as Docker containers rather than installing K3s directly on the host. That's why Docker is a hard prerequisite here.
2. **Write** `p3/scripts/install.sh`: installs Docker, then K3d (`curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash`), then `kubectl` if not already present.
3. **Create the cluster**: `k3d cluster create <name>`.
4. **Create namespaces**: `kubectl create ns argocd` and `kubectl create ns dev`.
5. **Install Argo CD** into the `argocd` namespace using its official install manifest (`kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`), then figure out how to reach its UI/API (port-forward is simplest to start with).
6. **Set up your public GitHub repo**: put your `p3` manifests there (at least a `deployment.yml` / `deployment.yaml` referencing the app image, plus service/ingress as needed), with a member's login in the repo name.
7. **Decide app source**: Wil's `wil42/playground` (fastest path) or your own two-tag image.
8. **Create an Argo CD** `Application` **resource** pointing to: your GitHub repo URL, the path with your manifests, target revision (e.g. `HEAD` /branch), and destination namespace `dev`.
9. **Sync and verify v1**: once synced, `curl http://localhost:8888/` (with port-forward or however you expose it) should report the v1 message/status.

10. **Switch version**: edit `deployment.yaml` in your GitHub repo to point at the `v2` image tag, commit + push.
11. **Trigger/observe the sync** in Argo CD (manual sync button, or wait for auto-sync if configured) and confirm the running pod changed.
12. **Verify v2**: `curl` again — response should now reflect v2. This whole change-version flow is exactly what will be demonstrated live during the defense, so practice it end to end.

Suggested overall order

1. Finish/validate **Part 1** (Vagrant + K3s server/agent) — you appear to already have this.
2. Finish/validate **Part 2** (single K3s VM + 3 apps + Ingress) — you appear to already have this too, worth re-testing against the checklist above.
3. Tackle **Part 3** from scratch: this is the newest territory for you (Docker, K3d, Argo CD, GitOps) — budget the most learning time here.

Submission checklist (from the subject)

- [ ] `p1/`, `p2/`, `p3/` folders at repo root, each with `scripts/` and `confs/` subfolders.
- [ ] No bonus folder needed (skipping it here) — but note: bonus is only graded if the mandatory part is 100% flawless.
- [ ] Everything reproducible from a clean VM using only the scripts/configs in the repo.
- [ ] Be ready to demo the Ingress in Part 2 and the v1→v2 GitOps update in Part 3 live.